

Lab 3: Architecture Style Analysis

Scenario: Self-Service Coffee Kiosk

1. Requirements and design challenges

Customers select Espresso, Americano, or Latte; choose Small or Large; pay by credit card; and receive a printed receipt. The kiosk uses a touch screen, receipt-printer hardware, and stored menu/pricing data. Key quality concerns are responsive and usable interaction, card-data security, reliable printing, and straightforward single-device operation.

2. Compare the architectural styles

Style	Advantages here	Trade-offs / risks	Fit for scenario
Layered	Separates UI, order rules, catalogue, and hardware/payment adapters; local interactions are simple and quick; interfaces support replacement.	One kiosk deployment can be a failure point; careless layer crossings create coupling; layers do not independently scale.	Best fit for a single, self-contained kiosk with modest menu data.
Microservices	Services may be deployed/scaled independently; boundaries can isolate faults and allow technology choice.	Requires deployment/orchestration, service discovery, observability, retries, and distributed consistency; adds network latency and operating burden.	Overkill for one kiosk; keep card processing external, rather than splitting local functions into services.
Client-Server	Central menu/pricing updates and data management suit a fleet of kiosks; server can serve many clients.	Requires connectivity/server availability; network round trips can slow interaction; server is a bottleneck and potential single failure point.	Potential future multi-kiosk model, but not required by the one-kiosk assignment.

3. Selection and component responsibilities

Select a layered architecture deployed locally as a modular kiosk application. Touchscreen UI owns presentation; Order Manager validates choices and coordinates the sale; Menu & Pricing Catalog supplies the six product/size combinations; Payment Service authorizes the card through the provider; Receipt Printer Adapter formats output and isolates device-driver details; Receipt Printer is the hardware component. The credit-card processor is external to the kiosk boundary.

4. Interfaces and interaction sequence

IOrderAPI: UI requires, Order Manager provides. IMenuCatalog: Order Manager requires, Catalog provides.

ICardAuthorization: Order Manager requires, Payment Service provides. IReceiptJob: Order Manager provides, Printer Adapter requires. IPrinter: Printer Adapter requires, printer hardware provides. Flow: choose item and size, validate price, authorize card, and, on approval, submit paid order details for printing. A declined payment must not generate a paid-sale receipt.

5. Security and performance implications

Use encrypted, authenticated processor communication and tokenization or hosted capture; never log or persist the card number/CVV; restrict device and payment credentials; comply with the processor's PCI controls. Local

menu/order handling avoids unnecessary network delay; the payment authorization is the one deliberate remote transaction, and printing goes directly through the local adapter.

Diagram key: <<component>> rectangles; green ball = provided interface; orange socket = required interface; solid line = assembly connector; dashed boundary = kiosk deployment/trust boundary.